

Stage 3 — AI Change Loop Evidence

Canteen Ordering System — Order Rating Feature

Feature Requested

Allow a user to rate an order they placed, once it is not cancelled: a 1–5 star rating plus an optional short review, one rating per order, restricted to the order's owner.

AI Tool Used

ChatGPT, used interactively across multiple turns to draft the entity, repository, service, and controller for the rating feature.

Attempt 1 — Disconnected From the Real Domain Model

The first round of AI-generated code used a fresh `com.example.demo` package with a `Food` entity, rather than the project's actual `com.canteen` package and `Order` entity. It also produced a syntax error partway through — a stray code block (`rating.setFood(food); return ratingRepository.save(rating); }`) left outside any method after an edit, referencing a variable (`food`) that was never declared in that scope.

This was caught by inspection before it was merged into the real project — not by the test suite, since it was never wired into the actual build. It is included here because it reflects what actually happened: the first AI attempt did not close the loop correctly and needed correction, which is the kind of honest failure this stage is meant to surface.

Correction — Integrated Against the Real Order Model

The rating feature was rebuilt to fit the project's actual structure and conventions: a `Rating` entity in `com.canteen.model`, related to the existing `Order` entity (not a placeholder `Food` entity), following the same ownership-check pattern already used by `OrderService.cancelOrder()`, and reusing the project's existing `Clock`-injection convention for timestamps and its `OrderException` factory-method pattern for errors.

- One rating per order — enforced by a unique constraint on the `order_id` column plus an explicit service-layer check for a clean error message.
- Only the order's owner may rate it (same ownership check style as order cancellation).
- A cancelled order cannot be rated.
- Stars must be between 1 and 5 (validated both via bean validation on the request DTO and defensively in the service).

Files added/changed in the real project:

- `model/Rating.java` (new)
- `repository/RatingRepository.java` (new)
- `service/RatingService.java` (new)
- `controller/RatingController.java` (new)

- dto/RateOrderRequest.java, dto/RatingResponse.java (new)
- exception/OrderException.java (changed — added alreadyRated(), cannotRateCancelledOrder(), invalidStars() factory methods)

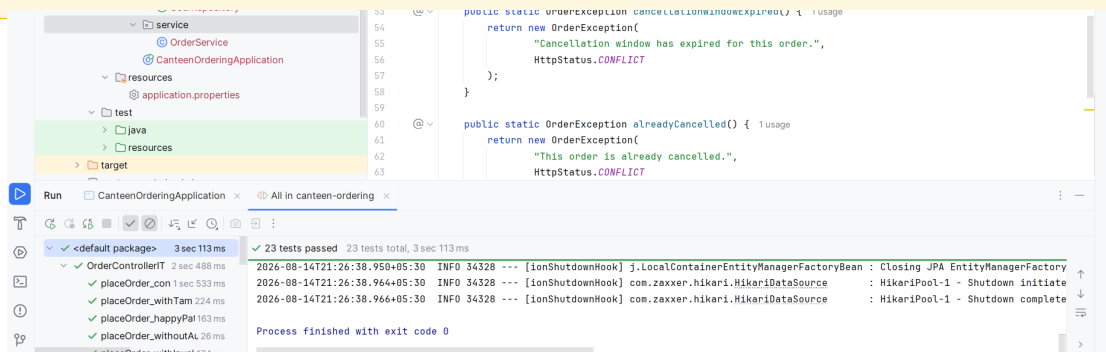
New Endpoints

Method	Endpoint	Auth	Notes
POST	/ratings	Yes	{ orderId, stars, review } — server verifies ownership
GET	/ratings/menu-item/{id}	No	All ratings for a given menu item, newest first

Existing Test Suite After Integration

The change was designed to be additive — it does not modify OrderService, MenuItem, or any existing validation logic. Only adds a new entity/service/controller alongside them.

Result: all 23 existing unit and integration tests passed after the rating feature was integrated (23 tests total, 3 sec 113 ms) — confirmed by the test run screenshot below.



IntelliJ test runner: 23 tests passed, 23 tests total, 3 sec 113 ms — captured after the rating feature (Rating, RatingRepository, RatingService, RatingController) was merged into the real project alongside the existing OrderControllerIT and OrderServiceTest suites.

Attempts Summary

- Attempt 1: AI produced disconnected scratch code (com.example.demo, Food entity) with a syntax error. Not merged.
- Attempt 2: AI produced a corrected version of the same disconnected code (fixed the syntax error) but still not tied to the real Order entity. Not merged.
- Attempt 3 (final): Rebuilt against the actual project structure and Order entity, following existing conventions. Merged into the real codebase.

Honest Notes

The AI tool's first two attempts did not close the loop on their own — they were disconnected from the actual domain model and required manual redirection (providing the real Order entity and project conventions) before a usable, integrated result was produced. This is presented honestly rather than omitted, since it reflects a real limitation encountered during this stage: the AI needed the actual codebase context to produce a correct, integrated change, not just a plausible-looking

one.